

Evaluating Machine Learning Models on a Modified MNIST Dataset



Machine Learning for Data Intensive Science

mem97

December 2024

Contents

1	Introduction	1
2	Background	1
2.1	MNIST	1
2.2	Neural Networks (NN)	1
2.3	Support Vector Machines (SVM)	2
2.4	Random Forest Classifiers	2
2.5	t-Distributed Stochastic Neighbour Embedding (t-SNE)	3
3	Generating 56×28 images	3
3.1	Justification	4
4	Fully Connected NN	5
4.1	model1	6
4.2	model2	6
4.3	model3	7
5	Random Forest & SVM	8
5.1	Random Forest	8
5.2	SVM	9
5.3	Comparison	9
6	Weak Linear Classifier	9
6.1	Setting A	9
6.2	Setting B	10
6.3	Comparing Setting A & B	11
7	Conclusion	11

Acknowledgment

I would like to acknowledge the invaluable assistance of ChatGPT-4 and Grammarly in the preparation of this project. The tools played a significant role in:

- Debugging and resolving coding challenges encountered during the development of the Python package.
- Providing insights and suggestions for improving the structure and functionality of the code.
- Generating concise and accurate summaries of complex texts to enhance understanding and clarity.

While ChatGPT-4 contributed significantly to streamlining the development process and improving the quality of outputs, all results were rigorously reviewed, tested, and refined to ensure their accuracy, relevance, and alignment with project objectives.

Word Count :3121

1 Introduction

This report aims to present the study on the MNIST dataset, (this contains 70000 images handwritten digits of dimension 28×28). In particular, training Neural Network models and others (SVM and Random Forests), to read images from a new dataset obtained from randomly allocating images from the original MNIST (new dataset 56×28 images of size 80000), the new labels for each image is the sum of the collection images. Additionally, we are going to analyse the performance of a weak linear classifier, on the 56×28 dataset compared to two sequential images from the original MNIST.

2 Background

This section aims to provide a short overview of the material covered in this project. In particular, what is the MNIST dataset? What are NN models? How do we train NN models to recognise had written digits from MNIST? What about SVM and Random Forsts?

2.1 MNIST

The MNIST[1, 8] dataset is a widely used benchmark dataset in machine learning (ML). This stands for **Modified National Institute of Standards and Technology**, which is a collection of handwritten digit images used for training and testing ML models, particularly in tasks like image classification and computer vision research.

Content, Structure and Pixel Values

MNIST dataset consists of 70000 grayscale images of handwritten digits from 0 to 9. Each image is size 28×28 pixels, making it 784 pixels in total. The dataset is divided into a **training set**, 60000 images, and a **test set**, 10000 images, moreover, each image is labelled with the digit it represents.

The pixel values range from 0 to 255, where 0 represents a white pixel (background), and 255 represents a black pixel (foreground). One can load the MNIST dataset using Python[5], in particular libraries like [tensorflow.keras](#) or [torchvision](#). For this project, [tensorflow.keras](#).

2.2 Neural Networks (NN)

A neural network[7] is a computational model inspired by the structure of neural networks found in human brain. This consists of layers of interconnected nodes (neurons), which process data by assigning weights and biases to inputs, and apply activation functions (e.g. Sigmoid, ELU, ReLU, Tanh) to generate outputs.

Basic Structure of NN

The first layer is the input layer, this receives the input features ($\vec{x} = x_1, x_2, \dots, x_n$). Next the hidden layers, the "magic" happens here. This is where the computations occur using weights, biases and activation functions. The number of hidden layers can massively affect the performance of a NN (as we will see when performing hyper-parameter tuning). The last layer is the output, which produces the final result.

Neuron Computation and Layer Outputs

Each neuron computes a weighted sum of inputs and pllies an activation function:

$$z = \sum_{i=1}^n w_i x_i + b \quad (1)$$

where x_i is the inputs to the neuron, w_i is the weights associated to each input and b is the bias term. The activation function f is applied to z to produce the neuron output, $a = f(z)$ where a is the output. For a layers with n inputs and m neurons we can extend Eq. (1) to:

$$z_j = \sum_{i=1}^n w_{ji} x_i + b_j \text{ for } j = 1, 2, \dots, m \quad (2)$$

Hence $a_j = f(z_j)$ for $j = 1, 2, \dots, m$. Here z_j is the weighted sum for neuron j , and a_j is the output of the neuron j after applying the activation function f .

Forward & Backward Propagation

For a NN with multiple layers, the forward propagation can be structured as follows:

- First layer: $z^{(1)} = W^{(1)}x + b^{(1)}, a^{(1)} = f(z^{(1)})$
- l^{th} layer: $z^{(l)} = W^{(l)}xa^{(l-1)} + b^{(l)}, a^{(l)} = f(z^{(l)})$

Where, $W^{(l)}$ is the weight matrix and $b^{(l)}$ is the bias vector for layer l and $a^{(l-1)}$ is the activation of the previous layer into the new one.

The backward propagation requires information about the loss function. The loss function defines the error between predictions and true values, two example of loss functions are the Mean Square Error and the Cross-Entropy Loss. The backward propagation adjusts weights and biases using the gradients to minimise loss functions:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial W^{(l)}} \quad (3)$$

One can compute the gradient of loss functions powerful algorithms such as Stochastic Gradient Descent (SGD)[9] and ADAM[3], both of these algorithms are already included in `tensorflow.keras.optimizers`.

2.3 Support Vector Machines (SVM)

A Support Vector Machine (SVM)[6] is a supervised ML algorithm used for classification and regression tasks. SVMs are particularly effective for high-dimensional spaces and cases where the number of dimensions exceeds the number of samples. The objective of an SVM is to find a hyperplane that best separates the data into different classes.

Characteristic of SVM

A hyperplane is a decision boundary that separates the data into different classes. For n -dimensional data, the hyperplane will have one less dimension i.e. $(n - 1)$. Hyperplanes are defined as:

$$\vec{w}^T \vec{x} + b = 0 \quad (4)$$

where $\vec{w}$ is the weight vector, $\vec{x}$ vector of data points and b is the bias term (determines the offset of the hyperplane from the origin).

2.4 Random Forest Classifiers

A Random Forest[2] is an ensemble learning method, just like SVM, this is used for classification and regression tasks. It operates by constructing a multitude of decision trees during training and outputs the class or mean prediction of the individual trees. Random Forests is considered to be a robust model in ML that handles large datasets and resistance to over-fitting.

Algorithm

For our specific purposes, we are only providing an overview of how the algorithm for classification is structured. The algorithm is divided into a training and prediction phase.

Training Phase:

1. Generate T bootstrapped datasets from the training data.
2. Train a decision tree for each dataset, where:
 - Only a subset of features is considered for splits.
 - Trees are grown to their maximum depth.

Prediction Phase:

1. Each tree makes a class prediction
2. The final class is decided by majority voting across all trees.

$$\hat{y} = \text{mode}(\{y_1, y_2, \dots, y_T\})$$

Both SVM and Random Forest Classifiers adopted using [scikit-learn](#) in Python.

2.5 t-Distributed Stochastic Neighbour Embedding (t-SNE)

t-SNE[4] is a non-linear dimensionality reduction technique used for visualisation datasets in 2 or 3 dimensions. It emphasises preserving local structures within the data, making it useful to explore complex datasets and clustering. One can utilise t-SNE on Python by importing the TSNE module from `sklearn.manifold`.

3 Generating 56×28 images

To generate 56×28 images, we first need to load the MNIST dataset and split it into training and testing sets:

```

1 from tensorflow.keras.datasets import mnist # importing 28x28 dataset
2
3 # loading the data splitting into x_train, y_train and x_test, y_test
4 (x_train, y_train), (x_test, y_test) = mnist.load_data()
5
6 # checing if data is loaded correctly
7 print('Train set shape images:', x_train.shape)
8 print('Test set shape images:', x_test.shape)
9 print('Train set shape label:', y_train.shape)
10 print('Test set shape label:', y_test.shape)

```

Output:

```

Train set shape images: (60000, 28, 28)
Test set shape images: (10000, 28, 28)
Train set shape label: (60000,)
Test set shape label: (10000,)

```

The MNIST data was successfully loaded, split into training sets `x_train` (images) and `y_train` (labels), and testing set `x_test` (images) and `y_test` (labels).

The next step is to randomly select elements from each set to generate a 56×28 dataset of total size 80000, this is then split into a training set of size 64000 and a test set of size 16000.

```

1 import numpy as np
2
3 np.random.seed(1) # fixed dataset
4 # Initialising parameters
5 # try
6 #total_size = 30000 # total dataset size
7 total_size = 0.8*10**5 # total dataset size
8 N_train = int(total_size*0.8) # training dataset size
9 N_test = int(total_size*0.2) # test dataset size
10
11 lenght_train = len(x_train)
12 lenght_test = len(x_test)
13
14 # Generating pairs of images for training and test dataset
15 x_train_pair = np.random.choice(np.arange(lenght_train), size = (N_train, 2))
16 x_test_pair = np.random.choice(np.arange(lenght_test), size = (N_test, 2))

```

```

17
18 # Summing the values of each images from training and test datasets
19 y_train_new = y_train[x_train_pair[:,0]] + y_train[x_train_pair[:,1]]
20 y_test_new = y_test[x_test_pair[:,0]] + y_test[x_test_pair[:,1]]
21
22 # Generating 56x28 images
23 x_train_new = np.append(x_train[x_train_pair[:,0]], x_train[x_train_pair[:,1]], axis=1)
24 x_test_new = np.append(x_test[x_test_pair[:,0]], x_test[x_test_pair[:,1]], axis=1)

```

Components of our new dataset:

- `x_train_new`, `x_test_new` 56×28 , i.e. random combination of two images from MNIST dataset.
- `y_train_new`, `y_test_new`, these labels are the sums of the coupled images generated.

3.1 Justification

The total size of the dataset, 80000, is large enough to represent the statistical properties of the MNIST dataset, ensuring adequate training and testing diversity. Furthermore, pairing images randomly ensures diverse combinations, avoiding bias, and promoting a robust model training.

Summing digit labels creates a new label range (0-18), where the distribution depends on the MNIST label distribution, preserving statistical properties across splits. Overall, this setup ensures a statistically representative dataset while maintaining methodological rigour for evaluating model performance.

Visualising the new dataset

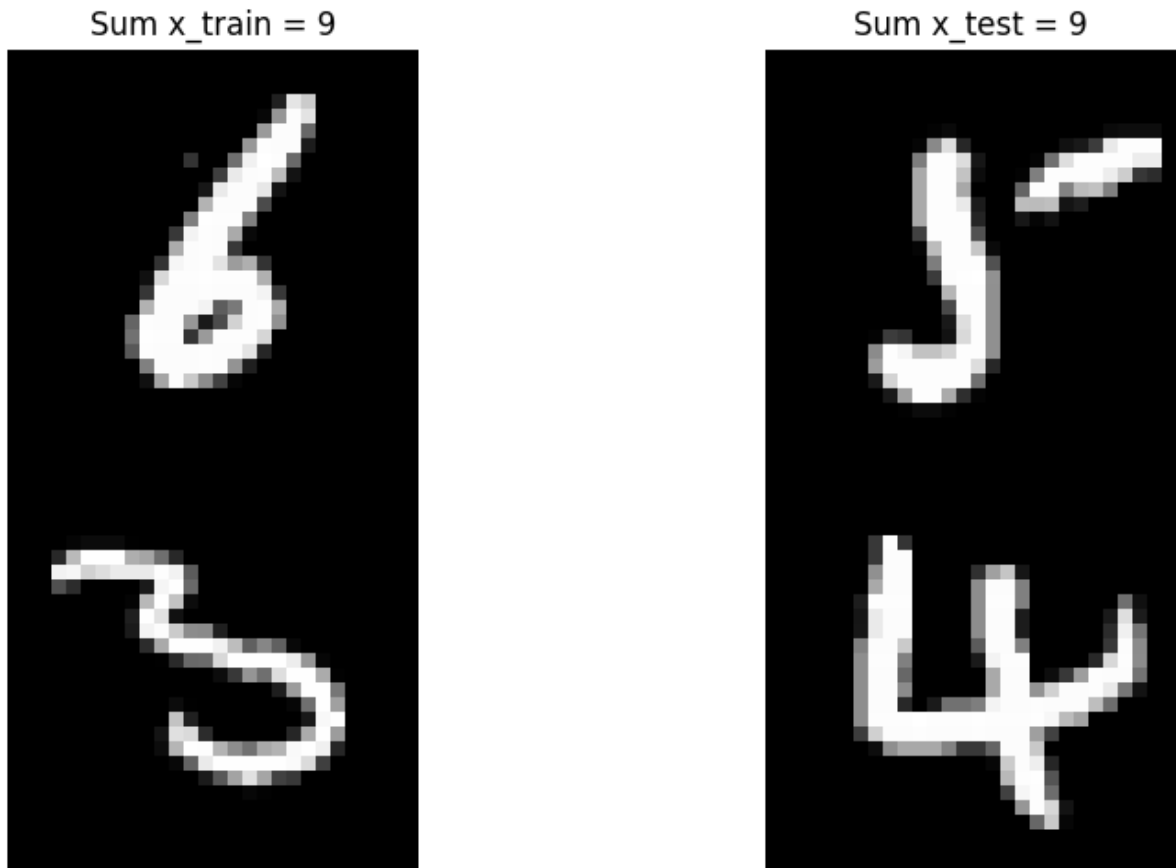


Figure 1: This image shows a random selected pair of images from the train and test sets with their respective labels

4 Fully Connected NN

This section focuses on training different fully connected NN, with different settings. I have trained three different models of NN, `model1`, `model2` and `model3` with the new generated dataset of 56×28 images. Each model differ from the other, aiming to determine which of them provides the best validation score. For all three models the following packages where used in the Jupyter Notebook:

```
1 import optuna
2 import numpy as np
3 import tensorflow as tf
4 import matplotlib.pyplot as plt
5
6 from tensorflow.keras.datasets import mnist #to import our dataset
7 from tensorflow.keras.models import Sequential, Model # imports our type of network
8 from tensorflow.keras.layers import Dense, Flatten, Dropout, Input, BatchNormalization
9 # imports our layers we want to use
10
11 from tensorflow.python.keras.losses import categorical_crossentropy #loss function
12 from tensorflow.keras.optimizers import Adam, SGD #optimisers
13 from tensorflow.keras.utils import to_categorical #some function for data preparation
14
15 from tensorflow.keras.callbacks import ModelCheckpoint #checkpoints used to keep track of
16 # best model
```

`optuna` is an open-source hyperparameter optimisation framework designed for efficiency and flexibility. It automates the search for optimal parameters using algorithms like Bayesian optimisation. Key features include pruning for early stopping, distributed optimisation, and a Pythonic interface.

All three models use `Sequential` from `tensorflow.keras.models`, `model1` is the simplest of the models, there is not hyper-parameter tuning performed for it. This is aimed to observe how well the tuned models perform compared to the simplest model.

Preparing Data for Models

Before initialising each model, the data needs to be normalised and reshaped. For all models, a batch¹ size of 128, number of classes (i.e. possible values 0-18) equal to 19 and 20 epochs².

```
1 batch_size = 2**7
2 num_classes = 19
3 epochs = 20
4
5 # input image dimensions
6 img_rows, img_cols = 56, 28
7 # normalising images
8 x_train_ = x_train_new.astype('float32')
9 x_test_ = x_test_new.astype('float32')
10 x_train_ /= 255 # max value for each pixel
11 x_test_ /= 255
12 print('x_train shape:', x_train_new.shape)
13 print(x_train_new.shape[0], 'train samples')
14 print(x_test_new.shape[0], 'test samples')
15
16 # convert class vectors to binary class matrices
17 y_train_flatten = to_categorical(y_train_new, num_classes)
18 y_test_flatten = to_categorical(y_test_new, num_classes)
```

¹A subset of the dataset used to train the model in a single forward and backward pass. It determines how many samples are processed before the model updates its weights.

²One complete iteration through the entire dataset during training, typically consisting of multiple batches.

4.1 model1

As mentioned, `model1` is the simplest of the model, with no hyper-parameter tuning. After 20 epochs the model achieved, a validation accuracy of 76.89%, this results is not terrible at all considering this is a simple NN model.

Output `model1`:

```
Epoch 20/20
500/500 3s 5ms/step - accuracy: 0.9235 - loss: 0.2679 - val_accuracy: 0.7689 - val_loss: 0.8626
```

Plot:

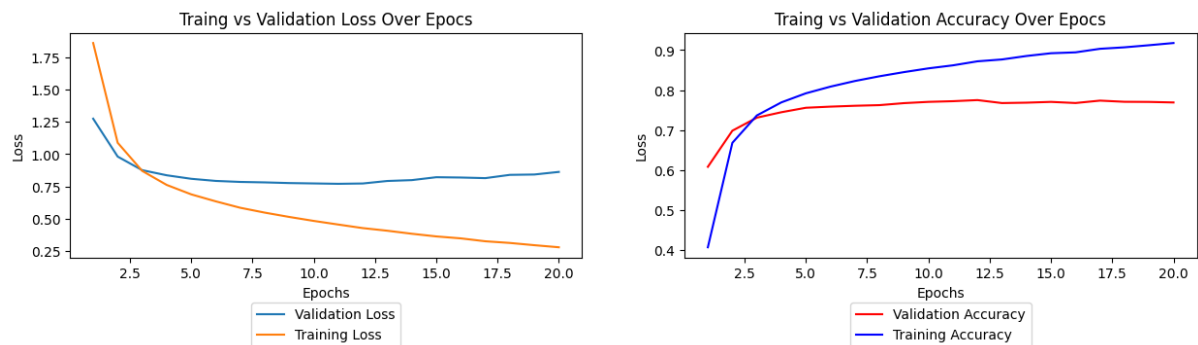


Figure 2: The plot on the left compares the training and validation loss, plot on the right training and validation accuracy both over number of epochs

From the above plot, we can observe that `model1` has overfitting issues, this is not just noticeable from the training and testing accuracy difference ($\approx 16\%$), but also from the massive spread one observes between the training and validation loss (left plot). To avoid overfitting in the other two models, we have implemented the following:

- Use regularisation (e.g. linear and ridge regularisation, often referred as L1 and L2)
- Dropout (disconnect random neurons)

4.2 model2

In this model we performed hyper-parameter tuning with `optuna`, (the code in the notebook was developed based on the code provided in one of the Jupyter Notebooks from the M1 lecture course, please see [Notebooks](#)). The hyper-parameter search was performed on:

- Number of layers range (1-3).
- Activation functions, options (`relu`, `sigmoid`, `tanh`, `softmax`)
- Dropout rate, range option (0%-80%)
- Number of neuron in each layer, ranging between (32-256)
- Learning rate, ranging between (10^{-5} , 10^{-2})

After 20 trial, the search stopped with output:

```
Best trial:
Value: 0.9003124833106995
Params:
  num_layers: 2
  activation: relu
```

```
dropout_rate: 0.13288286791395848
units: 236
learning_rate: 0.0036462592000901686
```

The result is promising, we can already see an improvement of ($\approx 14\%$) for the validation accuracy compared to `model11`. The above hyper-parameters, are implemented in `model12`, providing the following output:

```
Epoch 50/50
500/500 2s 3ms/step - accuracy: 0.9522 - loss: 0.1782 - val_accuracy: 0.9086 - val_loss: 0.5090
```

Plot: If we compare `model12` to our previous one, we can notice the difference between the training and

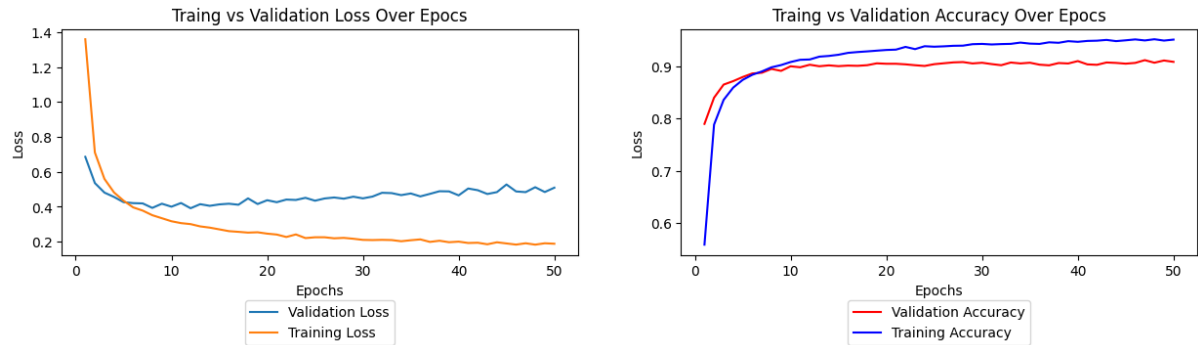


Figure 3: Same plots as before, but over 50 epochs instead

testing accuracy is now $\approx 5\%$, a significant improvement! But, when observing plot 3, it is clear that the model is still facing some overfitting issues.

4.3 model3

The same procedure as `model12` was applied here. The additional hyper-parameters implemented in the search are:

- Regularisation type, L_1 , L_2 and $L_1 + L_2$
- Regularisation strength, either for L_1 or L_2

After performing hyper-parameter search, the best result was provided when:

```
Best trial:
Value: 0.8973749876022339
Params:
  num_layers: 2
  activation: sigmoid
  dropout_rate: 0.0590731237212877
  units: 253
  learning_rate: 0.005196484442711773
  reg_type: l1
  reg_strength_l1: 2.06694264447087e-05
  reg_strength_l2: 0.0005409544812395567
```

The result obtained does not seem to differ too much from `model12` overall performance, but after training `model13` with the above hyper-parameters, this was the final output for 50 epochs:

```
Epoch 50/50
500/500 3s 5ms/step - accuracy: 0.9467 - loss: 0.5108 - val_accuracy: 0.9174 - val_loss: 0.6328
```

Plot:

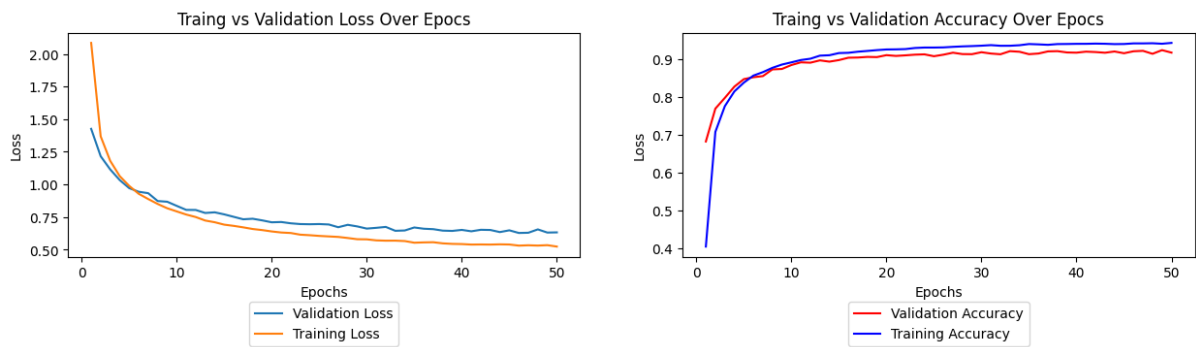


Figure 4: After further regularisation

There is no significant improvement on the validation accuracy compared to `model12` ($\approx 1\%$). However, it is clear from plot 4 that `model13` is not overfitting, this is what we wanted to accomplish.

Note! All models 1,2 and 3 have been saved in the folder `model_history_NN`, you can load them on the Notebook with the following commands:

```

1 from IPython.display import Image
2 load_best_model = tf.keras.models.load_model('model_history_NN/best_model.keras')
3
4 loss, accuracy = load_best_model.evaluate(x_test_, y_test_flatten)
5 Image('model_history_NN/plot_best_model.png')
```

5 Random Forest & SVM

To implement Random Forest and SVM on Python, one can import `RandomForestClassifier` from `sklearn.ensemble` and `SVC` from `sklearn.svm`. This section aims to provide a comparison between the two models, `optuna` was also implemented to determine the best results from each model. The best results are saved as `best_random_forest_model.pkl` and `best_svm_model.pkl` both saved as pickle files.

5.1 Random Forest

Preparing dataset

```

1 '''
2 RandomForestClassifier takes a feature matrix, say X, as a 2D array of shape (n_samples,
3 n_features).
4 From the previous question x_train_new and x_test_new are 3D arrays, hence, before
5 proceeding with RandomForestClassifier,
6 one needs to reshape x_train_new and x_test_new.
7 '''
8 # reshapeing x_train_new and x_test_new into 2D arrays
9 # for this task one can use .reshape
10 x_train_flattened = x_train_new.reshape(x_train_new.shape[0], -1)
11 x_test_flattened = x_test_new.reshape(x_test_new.shape[0], -1)
12 print('Checking if reshaping as been computed correctly:')
13 print('x_train_flattened shape', x_train_flattened.shape)
14 print('x_test_flattened shape', x_test_flattened.shape)
```

Hyper-parameter tuning after 50 runs:

```

Best hyperparameters: {'n_estimators': 194, 'max_depth': 23, 'min_samples_split': 6,
'min_samples_leaf': 1, 'max_features': 'sqrt'}
Test accuracy: 0.7791875
```

Overall best performance:

Visualising train and test score achieved from best model:

Score train: 0.999859375
Score test: 0.7791875

5.2 SVM

Preparing dataset

```
1 from sklearn.model_selection import train_test_split
2
3 # Use a smaller subset for optimization
4 X_train_subset, _, y_train_subset, _ = train_test_split(x_train_flattened, y_train_new,
5 test_size=0.8, random_state=1)
6 _, X_test_subset, _, y_test_subset = train_test_split(x_test_flattened, y_test_new,
7 test_size=0.2, random_state=1)
```

Since the SVM, requires a high computational power, this method is quite limited on regular computers, hence a small fraction of the overall dataset is used for hyper-parameter tuning, and then the results were implemented to train the SVM model over the whole dataset of 56×28 images.

Final output for SVM:

Score train: 0.963828125
Score test: 0.6575

5.3 Comparison

Both SVM and Random Forest models are overfitting, Random Forest seem to be the best model out of the two. However, SVM was limited, only a subsection of the overall dataset was utilised for training and testing, this does not facilitate the search for the best model with `optuna`. Additionally, many runs were stopped by `optuna` as it was exceeding the available memory of my machine, thus if the same study was performed with a high performance computer, we could see a very different outcome from the two methods.

6 Weak Linear Classifier

In this section we are comparing the performance of a weak linear classifier, trained on the 56×28 dataset with the probabilities of a single linear classifier which is applied on the two images sequentially. The training regime was repeated for samples of size 0,100,500, and 1000, to visualise how the performance changes.

Model adopted: Linear SVM with Minimal Margins.

Support Vector Machines (SVM) with a linear kernel can act as weak classifiers if the margin is too small or the dataset is not well-separated. Poor performance can result from weak regularization or insufficient data. `SVC` has a feature that allows to measure the probability for each number in the dataset to occur.

6.1 Setting A

A single linear classifier trained on the Combined Dataset

- A weak linear classifier is trained on the 56×28 dataset
- The dataset size $56 \times 28 = 1568$ features per sample.
- Probabilities are directly output by this classifier for test samples

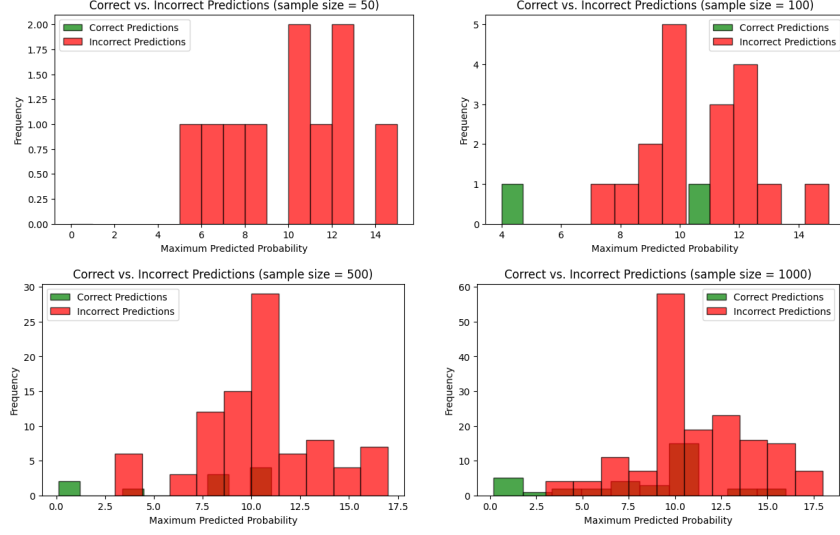


Figure 5: The above histograms represent the maximum probability count for each digit predicted from the 56×28 datasets.

6.2 Setting B

Sequential Application of single linear classifier

- Rather than training a classifier on the combined MNIST (56×28), train separate classifiers on two parts sequentially (i.e. first 28×28 , and then the second 28×28 dataset).
- The output of the two classifiers are combined (i.e., averaged probabilities or via secondary classifier) to make the final decision

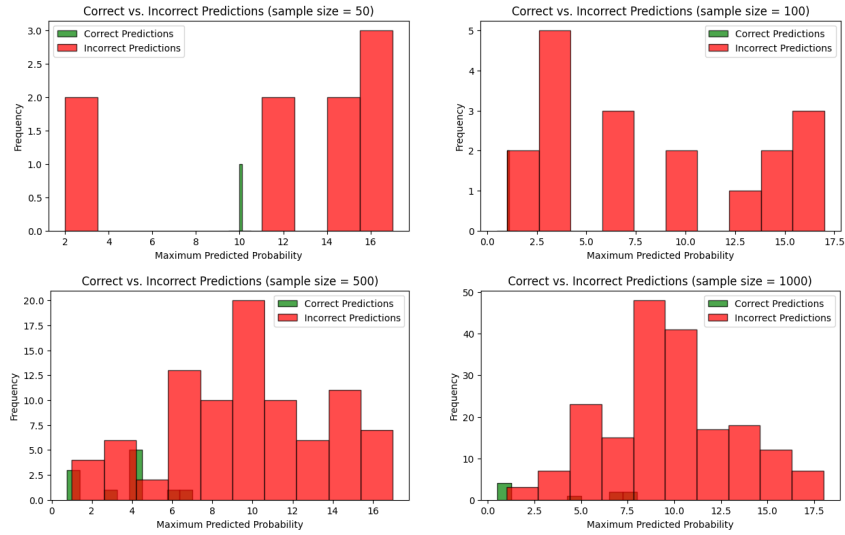


Figure 6: The above histograms represent the maximum probability count for each digit predicted from the $28 \times 28 + 28 \times 28$ datasets.

6.3 Comparing Setting A & B

The histograms illustrate in Figure (5) and (6) how confident the model is when making predictions and whether this confidence correlates with correctness. When correct predictions cluster at higher probabilities, it suggests good confidence calibration. Conversely, substantial overlap between correct and incorrect distributions at high probabilities indicates overconfidence and poor calibration. Differences between small and large sample sizes highlight how more training data can stabilize and sharpen the model's probability distribution. Comparing the combined dataset approach with the sequential approach provides insights into which training strategy leads to clearer separation of correct and incorrect predictions.

Accuracy Comparison

While training both models based on settings A and B, also the accuracy was collected and plotted. Training on separate 28×28 images rather than a combined 56×28 input often yields better results.

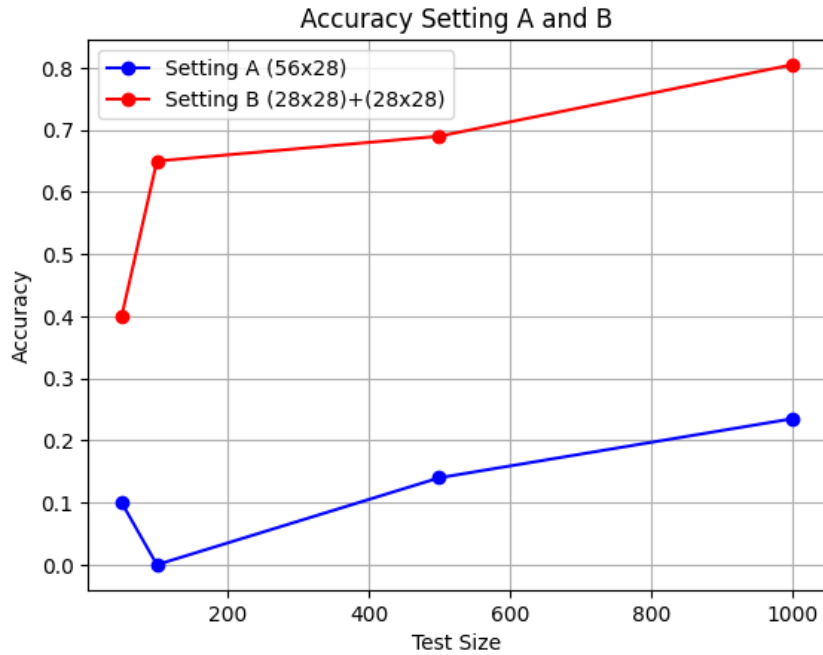


Figure 7: Comparing accuracies from both settings A and B.

Each 28×28 segment is simpler, matching standard MNIST data formatting and reducing complexity. When two specialized models are trained on these standard-sized inputs, they learn more direct, robust features. Combining their outputs leverages both classifiers' strengths. In contrast, a single model handling a larger, combined input deals with a more complicated feature space, often resulting in weaker performance and less accurate predictions.

7 Conclusion

This report explored the performance of several machine learning models on a modified MNIST dataset, including neural networks, Random Forests, SVMs, and weak linear classifiers. Fully connected neural networks achieved strong results, especially with hyper-parameter tuning, though overfitting remained an issue. Random Forests performed well, but SVMs were limited by computational constraints.

For weak linear classifiers, training on separate 28×28 images proved more effective than using combined 56×28 inputs. By simplifying the input, the models were able to learn clearer features, resulting in better accuracy and more reliable confidence scores. This highlights the importance of reducing complexity and using structured approaches when training models for robust performance.

References

- [1] Gregory Cohen et al. “EMNIST: Extending MNIST to handwritten letters”. In: *2017 international joint conference on neural networks (IJCNN)*. IEEE. 2017, pp. 2921–2926.
- [2] Mahesh Pal. “Random forest classifier for remote sensing classification”. In: *International journal of remote sensing* 26.1 (2005), pp. 217–222.
- [3] Sashank J Reddi, Satyen Kale, and Sanjiv Kumar. “On the convergence of adam and beyond”. In: *arXiv preprint arXiv:1904.09237* (2019).
- [4] Laurens Van der Maaten and Geoffrey Hinton. “Visualizing data using t-SNE.” In: *Journal of machine learning research* 9.11 (2008).
- [5] Guido Van Rossum and Fred L Drake Jr. *Python tutorial*. Vol. 620. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995.
- [6] Haifeng Wang and Dejin Hu. “Comparison of SVM and LS-SVM for regression”. In: *2005 International conference on neural networks and brain*. Vol. 1. IEEE. 2005, pp. 279–283.
- [7] Yu-chen Wu and Jun-wen Feng. “Development and application of artificial neural network”. In: *Wireless Personal Communications* 102 (2018), pp. 1645–1656.
- [8] Han Xiao, Kashif Rasul, and Roland Vollgraf. “Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms”. In: *arXiv preprint arXiv:1708.07747* (2017).
- [9] Sixin Zhang, Anna E Choromanska, and Yann LeCun. “Deep learning with elastic averaging SGD”. In: *Advances in neural information processing systems* 28 (2015).